

Programa · Comisión 2026

AI Engineering

Clase 5:

Razonamiento autónomo: introducción de agentes con
LangGraph

Objetivos

1. Estructurar flujos de IA con nodos, aristas y estado tipado usando LangGraph.
2. Diseñar ciclos de razonamiento que permitan a un agente reintentar, corregirse o escalar.
3. Definir y ejecutar herramientas (tools) seguras con Pydantic y LangChain.
4. Implementar el ciclo ReAct: razonar, actuar y observar antes de responder.
5. Configurar persistencia con Checkpointers para que un agente no pierda su memoria.

Agenda

1. Arquitectura de grafos: nodos, aristas y estado en LangGraph.
2. Tool Calling: definición y ejecución segura de herramientas.
3. Ciclos ReAct y persistencia de checkpoints en agentes autónomos.
4. Pre-Entrega 5: tu agente de razonamiento cíclico con memoria.

**¡Esta clase va a ser
grabada!**



01

Arquitectura de Grafos

¿Por Qué LangGraph?

De cadenas a grafos

Las cadenas de LangChain son excelentes para flujos predecibles: Input → Prompt → LLM → Output, pero son DAGs por naturaleza.

Un agente autónomo necesita ciclos: observar el resultado de una herramienta y decidir si terminó o debe intentar otra cosa.

LangGraph permite grafos con ciclos, y sobre todo, una gestión de estado que persiste mientras el agente navega por el flujo.

El Estado: Memoria Compartida

TypedDict como contrato

- El Estado es como el producto que se mueve por una cinta transportadora: cada nodo puede leerlo, modificarlo o añadirle piezas.
- Se define con TypedDict: un esquema claro y tipado de qué datos están "vivos" en la aplicación.
- `messages: Annotated[List[BaseMessage], add_messages]` es el campo más común, junto a campos propios como `is_escalated`.

Un nodo no sobrescribe todo el estado: devuelve solo una actualización parcial que LangGraph fusiona con el resto.

Reducers

`add_messages`, no sobrescribir

Un error común es pensar que un nodo sobrescribe todo el estado; en realidad, solo actualiza los campos que devuelve.

Si el estado tiene 10 campos y un nodo devuelve 1, solo ese campo cambia: el resto permanece intacto.

Un Reducer como `add_messages` le dice a LangGraph: "no borres los mensajes anteriores, anexá los nuevos a la lista".

Nodos: Unidades de Trabajo

Funciones testeables, no cajas negras

Los Nodos son funciones de Python (síncronas o async) que toman el estado actual y devuelven una actualización.

Un nodo no tiene por qué ser un LLM: puede ser una herramienta, una consulta a una API, o lógica de negocio pura.

Anatomía: recuperar datos del estado, realizar la operación, devolver la actualización. LangGraph fusiona el resto.

Principio de diseño: nodos pequeños con una única responsabilidad, fáciles de testear y observar.

Aristas: Control de Flujo

Aristas Directas

Conectan el Nodo A con el Nodo B sin condiciones: siempre que termines en A, andá a B.

Aristas Condicionales

Bifurcan el flujo según una función de enrutamiento que lee el estado actual.

Ciclos de Reflexión

Una arista condicional puede volver a un nodo anterior: ahí nace la autonomía del agente.

El Grafo Compilado

Elegí tu propia aventura

Al compilar, LangGraph valida que exista un punto de entrada (START) y que todos los caminos lleguen a END, sin nodos aislados.

Metáfora: cada nodo es una página de un libro "Elige tu propia aventura"; el Estado es lo que el lector recuerda.

Las aristas condicionales son instrucciones basadas en el inventario: "si tenés la llave dorada, andá a la página 100".

Errores Comunes

Tres trampas al diseñar un grafo

Cargar el estado con datos innecesarios: clientes de bases de datos o conexiones de red no deberían vivir en el State.

Nodos demasiado grandes: si un nodo llama al LLM, procesa, limpia y guarda en la base, perdés la granularidad del grafo.

Olvidar el Reducer: sin `Annotated[List[BaseMessage], add_messages]`, cada nodo borra el historial anterior.

Ciclo de Reintento

search → retry → escalate

Un grafo mínimo busca una solución, reintentará si no la encuentra, y escala a un humano tras 3 intentos.

La arista condicional route lee el estado: si no está resuelto y no llegó a 3 intentos, vuelve a "search" (retry): ese retorno es el ciclo.

Si nunca resuelve, al llegar a 3 intentos la ruta es "escalate" hacia el nodo humano, y termina en END.

Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

Pensá en el bucle de reflexión escritor↔crítico: el escritor mejora en cada vuelta hasta llegar a un score de 8. ¿Qué pasaría sin un tope de revisiones?

Práctica - Unidad 1

Diagramá tu grafo cíclico

Objetivo: diagramar un grafo de soporte con ciclo y escalado, y el bucle de reflexión escritor↔crítico.

Dibujá los nodos, las aristas condicionales, y marcá con una flecha el "retry" que vuelve hacia atrás: eso es el ciclo.

- Resultado: un diagrama que muestra por qué LangGraph no es una cadena lineal.
-

02

Tool Calling

¿Qué es Tool Calling?

Manos para un cerebro brillante

El modelo no ejecuta código directamente: genera una solicitud estructurada (JSON) indicando qué función quiere ejecutar y con qué parámetros.

Tu aplicación intercepta esa solicitud, ejecuta el código real, y le devuelve el resultado al modelo.

Es lo que le da "manos" a un LLM que, sin herramientas, solo puede razonar puertas adentro.

El Contrato

Nombre y Descripción

Un identificador único, y una explicación semántica: el LLM decide si usar la herramienta según esto.

Esquema de Entrada

Definición estricta de variables: tipos y obligatoriedad, validada con Pydantic.

Esquema de Salida

Qué forma tiene el resultado que la herramienta le devuelve al modelo.

Definición Segura

Pydantic antes que tu infraestructura

No podemos confiar en que el LLM envíe siempre los datos correctos: Pydantic valida los tipos antes de que el código toque tu infraestructura.

`@tool(args_schema=CalculatorInput)` conecta un esquema Pydantic con una función de Python decorada como herramienta.

La función sigue siendo Python puro: el decorador solo la expone como algo que el LLM puede invocar.

El Ciclo de Ejecución

Reason → Act → Observe

Reasoning: el LLM recibe el prompt y las definiciones de herramientas, y decide que necesita usar una.

- **Tool Call:** el LLM responde con un objeto estructurado indicando el nombre de la función y sus argumentos.
- **Execution:** tu servidor detecta la intención, valida los argumentos con Pydantic, y ejecuta la función real.
- **Observation:** el resultado se envía de vuelta al LLM en un nuevo mensaje con rol "tool".

Final Response: el LLM procesa el resultado y responde al usuario en lenguaje natural.

Errores y Seguridad

Las manos necesitan límites

Confundir Invocación con Ejecución: el LLM no ejecuta código; tu arquitectura debe dar permisos limitados a cada herramienta.

Nunca permitas que una herramienta ejecute SQL crudo generado por el LLM sin validación.

- Descripciones Ambiguas: "Busca datos" no alcanza. Sé específico sobre qué busca y dónde.

Ignorar Errores: si la herramienta falla, captúralo y envíalo al LLM como string para que lo explique o intente otra estrategia.

Momentos de la Clase:

Para recordar

Coder Tips

Material ampliado

Cuestionario de tarea

#FindTheBug

La palabra clave del día es Least
Privilege: cada herramienta
debería poder hacer lo mínimo
necesario, ni un permiso más.

Práctica - Unidad 2

Tu primera herramienta

Objetivo: definir una herramienta segura con Pydantic y LangChain, lista para que un LLM la invoque.

Manos a la obra:

- Definí un esquema de entrada con Pydantic, con tipos y descripciones claras para cada campo.
- Decorá tu función con `@tool(args_schema=...)` y escribí un docstring bien específico.

Resultado: una herramienta que el modelo puede elegir usar, con argumentos validados antes de ejecutarse.

Break,

andá por un cafecito
o mate, ¡y volvemos!



03

Ciclos ReAct y Checkpoints

El Ciclo ReAct

Reasoning + Acting

Pensar (Reason): el LLM analiza la solicitud del usuario y decide qué necesita.

Actuar y Observar: selecciona una herramienta, genera los parámetros, y recibe la salida para re-evaluar el estado.

Es cíclico: el LLM puede decidir usar tres herramientas distintas antes de dar una respuesta final.

Checkpoints

thread_id

Si el proceso se detiene, lo reanudás pasando el mismo thread_id: LangGraph carga el último estado conocido.

Time Travel

Los checkpoints permiten volver atrás en el tiempo para debuggear, o pedir aprobación humana antes de un paso crítico.

Sin Persistencia

Un agente efímero pierde todo el razonamiento y los tokens gastados si falla a mitad de una ejecución larga.

El Agente Climatológico

El enunciado de la unidad

Vas a construir un agente que razona si necesita herramientas y no pierde la memoria aunque interrumpas el script.

Debe usar `Annotated[list[BaseMessage], add_messages]` para que los mensajes se concatenen, no se sobrescriban.

El grafo termina solo cuando el LLM decide que no hay más herramientas que llamar.

El ToolNode debe mapear sus resultados de vuelta al agente, cerrando el ciclo ReAct.



Práctica - Unidad 3

Memoria entre mensajes

Objetivo: implementar un ciclo ReAct completo con persistencia en memoria.

Manos a la obra:

- Configuraré un checkpoint y probaré que el agente recuerde la ubicación mencionada en el primer mensaje, con el mismo `thread_id`.
- Definiré un `recursion_limit` para evitar bucles infinitos si el agente nunca decide terminar.

Resultado: un agente climatológico que razona, actúa, observa y no olvida.

PE

Pre-Entrega 5

¿Qué Debes Construir?

Pre-Entrega 5

Autonomía: el agente decide por sí mismo cuándo llamar a una herramienta, sin rutas manuales if/else.

- **Ciclo de Retorno:** si una herramienta falla o devuelve información incompleta, el agente reintenta o pide aclaraciones.
- **Resiliencia de Estado:** con un `thread_id`, el agente recuerda interacciones previas dentro de la misma sesión.

Código Limpio: Python 3.12, tipado estático y gestión asíncrona con `asyncio`.

Tu Misión

El entregable

 **Entrega: repositorio con el StateGraph, una herramienta propia, persistencia (SqliteSaver) y una traza ReAct en log o .json**

 **No hace falta un documento aparte: la traza del razonamiento se entrega dentro del repo**

Construí un agente que muestre, en un log, cómo razonó, qué herramienta llamó y qué observó antes de responder.

Pasos

1. Inicializá un proyecto Python 3.12+ con Poetry o venv.
2. Definí un StateGraph que herede de MessagesState.
3. Creá al menos una herramienta personalizada que simule una base de datos o búsqueda técnica.
4. Configurá un LLM vinculado a las herramientas con `llm.bind_tools()`.
5. Implementá persistencia con `SqliteSaver` y probá una ejecución con razonamiento multi-paso.

Errores a Evitar

Antes de entregar, revisá esto

Descripciones Vagas: si el agente no usa la herramienta que esperás, el error suele estar en el docstring, no en la lógica del grafo.

- Bucles Infinitos: definí siempre un recursion_limit (por ejemplo, 10 pasos) para evitar costos inesperados en la API.
- Estado Sucio: el estado se acumula; limpiá o resumí mensajes si el contexto se vuelve demasiado grande.
- No incluir las API Keys en el repositorio: usá siempre variables de entorno.

Olvidar el README: debe explicar cómo levantar el entorno y mostrar un ejemplo de traza de ejecución.

No probar el camino de fallo: verificá qué pasa si una herramienta devuelve un error o timeout.

¿Cómo se evalúa?

· 100 pts

Arquitectura y Estado · 30%

Correcta implementación del StateGraph y uso de MessagesState para mantener la coherencia del diálogo.

Herramientas y Razonamiento · 35%

Capacidad del agente para invocar herramientas externas y procesar información en múltiples iteraciones.

Persistencia y Calidad · 35%

Mecanismos de memoria con SqliteSaver, y estándares de la industria en la organización del repositorio.

Síntesis del Módulo

De cadenas a grafos con memoria

Pasamos de cadenas lineales a grafos con estado, nodos y aristas que permiten ciclos de razonamiento real.

- **El Estado es la memoria compartida; los Nodos transforman esa memoria; las Aristas deciden hacia dónde sigue el flujo.**
- **Tool Calling le da manos a tu agente, y los Checkpoints le dan memoria que sobrevive a un reinicio.**

Próximo módulo: de un agente que razona solo, a varios agentes especializados que colaboran entre sí.

Debugging y Ciclos en Vivo

Sesión en Vivo 5

Repasaremos en vivo cómo depurar un grafo que entra en un ciclo infinito, usando Time Travel para inspeccionar cada checkpoint.

- **Live Coding: convertiremos una cadena lineal en un grafo con al menos un ciclo de reintento, en tiempo real.**
- **Resolveremos dudas puntuales sobre tu Pre-Entrega 5 antes de la entrega final.**

Traé tu StateGraph, aunque esté incompleto: vamos a iterar sobre código real, no solo teoría.

¿Preguntas?

De cadenas lineales a grafos que razonan. Soltá tus dudas del módulo.



Glosario

1. LangGraph: framework de LangChain para construir grafos de estado con ciclos.
2. StateGraph: la clase que define nodos, aristas y el esquema de estado de un grafo.
3. TypedDict: estructura de Python que define el esquema tipado del estado.
4. Reducer: función que decide cómo se combina una actualización con el estado existente.
5. add_messages: reducer estándar que concatena mensajes nuevos en vez de sobrescribirlos.
6. Nodo: función de Python que lee el estado y devuelve una actualización parcial.
7. Arista Directa: conexión fija entre dos nodos, sin condición de por medio.

1. Arista Condicional: conexión cuyo destino depende de una función de enrutamiento sobre el estado.
2. START / END: los nodos especiales que marcan el punto de entrada y las salidas válidas de un grafo.
3. Ciclo: una arista que vuelve a un nodo anterior, permitiendo reintentos o reflexión.
4. Tool Calling: mecanismo por el cual un LLM solicita ejecutar una función externa de forma estructurada.
5. Contrato de Herramienta: nombre, descripción y esquemas de entrada/salida que definen una tool.
6. @tool: decorador de LangChain que expone una función de Python como herramienta invocable.
7. Least Privilege: principio de darle a cada herramienta solo los permisos mínimos necesarios.

1. ReAct (Reason + Act): patrón donde el agente razona, actúa con una herramienta y observa el resultado en ciclo.
2. ToolNode: nodo de LangGraph especializado en ejecutar las herramientas solicitadas por el LLM.
3. tools_condition: arista condicional predefinida que decide si el flujo va a herramientas o termina.
4. bind_tools: método que vincula un conjunto de herramientas a un LLM para que pueda invocarlas.
5. Checkpointer: mecanismo que guarda una fotografía del estado del grafo tras cada nodo ejecutado.
6. thread_id: identificador de sesión que permite recuperar el estado guardado de una conversación.
7. Time Travel / Human-in-the-loop: volver a un checkpoint anterior, o pausar el grafo para aprobación humana.

¡Gracias, arquitectos de IA!

We have provided this presentation to you for informational and illustrative purposes on a confidential basis. We reserve the right to use of non-disclosure agreements (NDAs) to protect our privacy and intellectual property rights. The information contained in this document is highly sensitive, confidential and/or proprietary and is intended for the express use of the intended recipient, as denoted on the title page of this document. The recipient of this presentation agrees by its receipt not to reproduce, duplicate, or reveal, in whole or in part, information presented herein without written permission of Coderhouse. No representation or warranty, expressed or implied, is made as to the accuracy or completeness of the information contained in this presentation. Any quote contained herein is for estimation purposes. Estimates are based on current, known requirements. Actual estimates may change once project elements are finalized or negotiated. This document may contain confidential pricing information. All pricing is subject to change.

Copyright © 2026 Coderhouse. All rights reserved. Coderhouse and the Coderhouse logo are trademarks or registered trademarks of Coderhouse, in the U.S. and/or other countries. All other trademarks are the property of their respective owners.